

PROGRAMAÇÃO MODULAR

MODULARIDADE:
COESÃO, ACOPLAMENTO,
ABSTRAÇÃO E ENCAPSULAMENTO

PROF. JOÃO CARAM

O QUE JÁ SABEMOS: MODULARIDADE

2

“Mecanismo para aumentar a flexibilidade e compreensibilidade de um sistema, em conjunto com a redução do seu tempo de desenvolvimento.”

(Parnas, David L. On the Criteria To Be Used in Decomposing Systems into Modules. Communications of the ACM, Vol. 15, No. 12, pp. 1053 – 1058, 1972. Tradução livre do autor.)

O QUE JÁ SABEMOS: POO

3

“I’m sorry that I long ago coined the term ‘objects’ for this topic because it gets many people to focus on the lesser idea. The big idea is ‘messaging’.”
(Alan Kay¹)

1 – Pai dos conceitos de POO, GUI, notebooks e da linguagem Smalltalk. Guitarrista e compositor profissional de jazz, cenógrafo profissional e organista clássico amador

O QUE JÁ SABEMOS: P00

4

- P00, segundo Alan Kay:
 - troca de mensagens;
 - proteção e retenção locais possibilitando ocultar o estado ou o processo;
 - associação tardia e dinâmica de tudo o que for possível.

O QUE JÁ SABEMOS: POO

5

- Análise orientada por objetos



- Projeto orientado por objetos

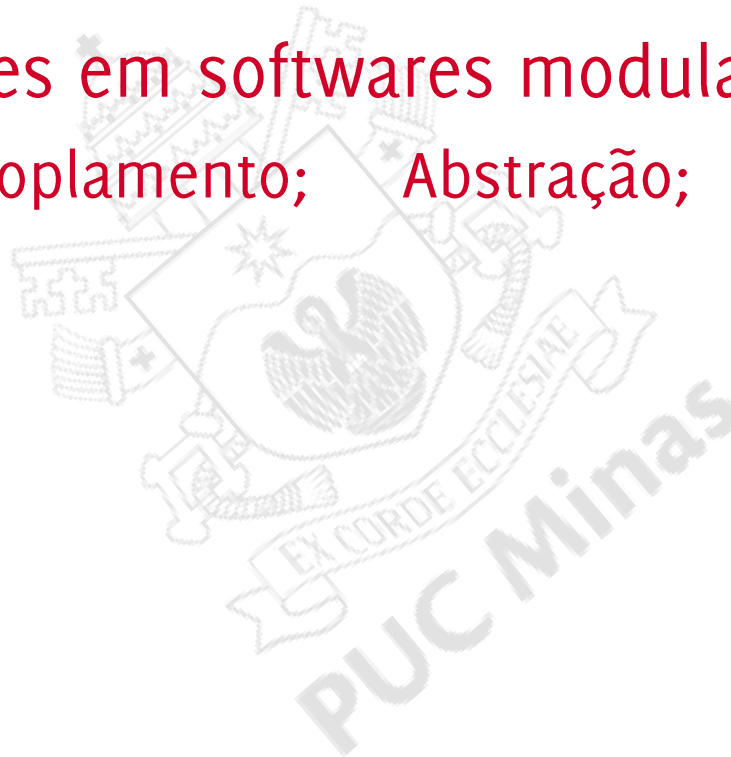


- Programação orientada por objetos

PILARES DA MODULARIDADE

6

- Quatro pilares em softwares modulares:
Coesão; Acoplamento; Abstração; Encapsulamento.



MODULARIDADE - COESÃO



COESÃO

8

- Coesão: Qualidade de uma coisa em que todas as partes estão ligadas umas às outras¹.
- Objetivo de um módulo em P00:
 - alta coesão interna, dependência intramodular;
 - utilizar informações internas de forma coerente para resolver um problema específico;
 - princípio da responsabilidade única (SRP).

¹ - Dicionário Priberam da Língua Portuguesa, 2008-2022. Disponível em <<https://dicionario.priberam.org/coes%C3%A3o>>, acessado em Fev/2022

COESÃO

9

- Coesão: Qualidade de uma coisa em que todas as partes estão ligadas umas às outras¹.
- Sua classe:
 - tem atributos que não estão sendo usados por nenhum método?
 - tem métodos que não usam atributos da classe?
- Se sim, problemas...

ALTA COESÃO E PROJETO OO

10

- ▣ Princípio da responsabilidade única.

- ▣ P00, segundo Alan Kay:



- ▣ troca de mensagens;



- ▣ proteção e retenção locais possibilitando ocultar o estado ou o processo;

- ▣ associação tardia e dinâmica de tudo o que for possível.

MODULARIDADE - ACOPLAMENTO



INDEPENDÊNCIA FUNCIONAL

12

- Módulo: “grupo de comandos com uma função bem definida e o mais independente possível em relação ao resto do algoritmo.”
- A dependência pode ser medida pela quantidade de conexões entre os elementos de software.

ACOPLAMENTO

13

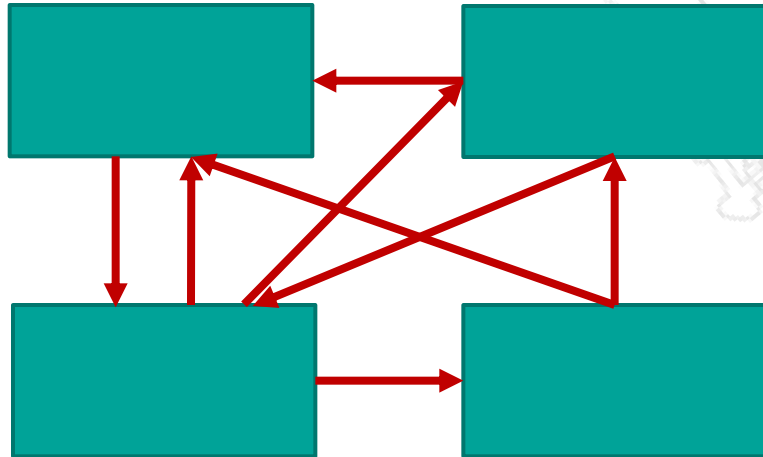
- Medida da interconexão entre elementos de software.



ACOPLAMENTO

14

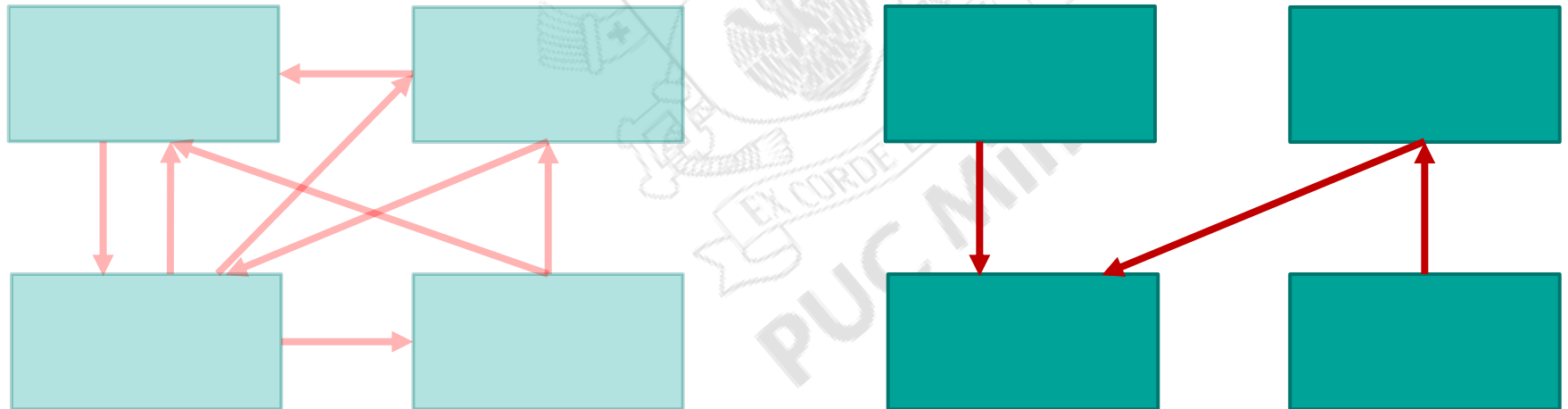
- "Espaguete": alto acoplamento.



ACOPLAMENTO

15

- Situação ideal: baixo acoplamento.



ACOPLAMENTO

16

- Acoplamento: medida da interconexão entre classes.
- Sua classe:
 - recebe muitos parâmetros de outra classe?
 - precisa receber o mesmo parâmetro em vários métodos?
 - depende unicamente de atributos e métodos de outras classes para funcionar?
- Se sim, problemas...

BAIXO ACOPLAMENTO: INDICADORES

17

- Tamanho:
 - Menor quantidade de parâmetros e métodos públicos.
- Visibilidade:
 - Parâmetros quando necessário x variáveis globais.
- Flexibilidade:
 - Mais facilidade na chamada (abordaremos no futuro).

BAIXO ACOPLAMENTO E PROJETO OO

18

▣ P00, segundo Alan Kay:



▣ troca de mensagens;



▣ proteção e retenção locais possibilitando ocultar o estado ou o processo;

▣ associação tardia e dinâmica de tudo o que for possível.

MODULARIDADE - ABSTRAÇÃO



UMA CLASSE HORA

20

- Projete e implemente uma classe Hora para ser utilizada em sistemas diversos.
 - A hora será representada por minutos e segundos;
 - Um objeto Hora só pode armazenar estados válidos.
 - Um objeto Hora pode ser incrementado em minutos, mantendo seu estado válido.
 - Um objeto Hora pode ser comparado com outro para verificação de qual horário está mais adiante.

QUEREMOS VER CÓDIGO

21

- Mas antes do código...



QUEREMOS VER CÓDIGO

22

- Mas antes do código...

- Análise

- Projeto



PENSANDO EM CLASSES

23

- ▣ Hora com hora e minuto; validação, incremento e comparação com outra hora.
- ▣ Atributos:
 - hora, minuto (int)
- ▣ Métodos:
 - incrementar(int quantidade): Hora
 - ajustar(): void
 - estahNaFrenteDe(Hora outra): boolean
 - ...

CLASSE HORA: UMA VERSÃO

Hora até segundos; validação, incremento e comparações.

Hora
-horas: int -minutos: int
+Hora(horas:int,minutos:int) -estahValida(): boolean -ajustar(): void +estahNaFrenteDe(outra:Hora): boolean +incrementar(quantidade:int): Hora +horaFormatada(): String

CLASSE HORA: UMA VERSÃO

Hora até segundos; validação, incremento e comparações.

Hora
-horas: int -minutos: int
+Hora(horas:int,minutos:int) -estahValida(): boolean -ajustar(): void +estahNaFrenteDe(outra:Hora): bool +incrementar(quantidade:int): Hora +horaFormatada(): String

Você
pensou
nisso?

USANDO A CLASSE HORA

```
class Aplicacao {  
    public static void main(String args[]){  
        Hora hora1 = new Hora(7,26);  
        Hora hora2 = new Hora(10,1);  
        String msg = "Hora2 na frente de Hora1";  
        boolean naFrente = hora1.estahNaFrenteDe(hora2);  
        if(naFrente)  
            msg = "Hora1 na frente de Hora2";  
        System.out.println(msg);  
        hora2.incrementar(100);  
        naFrente = hora1.estahNaFrenteDe(hora2);  
        System.out.println("Hora: "+hora1.horaFormatada());  
    }  
}
```

USANDO A CLASSE HORA

```
class Aplicacao {  
    public static void main(String args[]){  
        Hora hora1 = new Hora(7,26);  
        Hora hora2 = new Hora(10,1);  
        String msg = "Hora2 na frente de Hora1";  
        boolean naFrente = hora1.estahNaFrenteDe(hora2);  
        if(naFrente)  
            msg = "Hora1 na frente de Hora2";  
        System.out.println(msg);  
        hora2.incrementar(100);  
        naFrente = hora1.estahNaFrenteDe(hora2);  
        System.out.println("Hora: "+hora1.horaFormatada());  
    }  
}
```

USANDO A CLASSE HORA

```
class Aplicacao {  
    public static void main(String args[]){  
        Hora hora1 = new Hora(7,26);  
        Hora hora2 = new Hora(10,1);  
        String msg = "Hora2 na frente de Hora1";  
        boolean naFrente = hora1.estahNaFrenteDe(hora2);  
        if(naFrente)  
            msg = "Hora1 na frente de Hora2";  
        System.out.println(msg);  
        hora2.incrementar(100);  
        naFrente = hora1.estahNaFrenteDe(hora2);  
        System.out.println("Hora: "+hora1.horaFormatada());  
    }  
}
```

USANDO A CLASSE HORA

```
class Aplicacao {  
    public static void main(String args[]){  
        Hora hora1 = new Hora(7,26);  
        Hora hora2 = new Hora(10,1);  
        String msg = "Hora2 na frente de Hora1";  
        boolean naFrente = hora1.estaNaFrenteDe(hora2);  
        if(naFrente)  
            msg = "Hora1 na frente de Hora2";  
        System.out.println(msg);  
        hora2.incrementar(100);  
        naFrente = hora1.estaNaFrenteDe(hora2);  
        System.out.println("Hora: "+hora1.horaFormatada());  
    }  
}
```

USANDO A CLASSE HORA

```
class Aplicacao {  
    public static void main(String args[]){  
        Hora hora1 = new Hora(7,26);  
        Hora hora2 = new Hora(10,1);  
        String msg = "Hora2 na frente de Hora1";  
        if(hora1.estaNaFrenteDe(hora2))  
            msg = "Hora1 na frente de Hora2";  
        System.out.println(msg);  
        hora2.incrementar(100);  
        naFrente = hora1.estaNaFrenteDe(hora2);  
        System.out.println("Hora: "+hora1.horaFormatada());  
    }  
}
```

HORA E ABSTRAÇÃO

31

▣ Quem define como é modelada a hora?

- ▣ Int ou byte?
- ▣ Segundos, milésimos?
- ▣ Só incrementa, não decrementa?
- ▣ Mostra a diferença em segundos entre dois objetos '*Hora*'?

Hora
-horas: int -minutos: int
+Hora(horas:int,minutos:int) -estahValida(): boolean -ajustar(): void +estahNaFrenteDe(outra:Hora): boolean +incrementar(quantidade:int): Hora +horaFormatada(): String

PIZZA E ABSTRAÇÃO

32

- Quem define como é modelada a pizza?
 - Int ou byte para ingredientes?
 - Dimensão / fatias da pizza?
 - Não tem lista de ingredientes?
 - Não pode retirar ingredientes?

Pizza
- maxIngredientes: int - precoBase: double - valorPorAdicional: double - quantidadeIngredientes: int
+ Pizza() + Pizza(adicionais: int) - init(adicionais: int) : void - valorAdicionais() : double - podeAdicionarr(quantos: int) : boolean + valorFinal() : double + adicionarIngredientes(quantos: int) : int + notaDeCompra() : String

ABSTRAÇÃO

33

“Capacidade de enxergar uma operação complexa de uma forma simplificada.”

“Visualização ou representação de uma entidade que inclui somente os atributos de importância em um contexto particular.” (Sebesta, 2000)

ABSTRAÇÃO: REPRESENTAÇÃO

34

- Um módulo deve prover uma boa abstração da função pela qual é responsável:
 - Pessoa → cadastro de cidadãos (identidade);
 - Pessoa → cadastro acadêmico;
 - Pessoa → cadastro na *Epic Store*.
- Além disso...

ABSTRAÇÃO: OPERAÇÃO SIMPLIFICADA

35

- Conceito da caixa-preta:
 - Entrada e saída bem conhecidas;
 - Detalhes ocultos.
- Ideia principal: não é necessário saber detalhes do funcionamento de um objeto para utilizá-lo.

ABSTRAÇÃO: OPERAÇÃO SIMPLIFICADA

36

- Caixa-preta e abstração:
 - Como o SGA verifica seus pré-requisitos?
 - Como funciona a autenticação compartilhada da rede sem fio e o aplicativo acadêmico?
 - Como um áudio pode ser escutado em velocidade 2x?

ABSTRAÇÃO E PROJETO OO

37

- ▣ P00, segundo Alan Kay:
 - ▣ troca de mensagens;
 - ➡ ▣ proteção e retenção locais possibilitando ocultar o estado ou o processo;
 - ➡ ▣ associação tardia e dinâmica de tudo o que for possível.

MODULARIDADE - ENCAPSULAMENTO



ENCAPSULAMENTO

39

- Encapsular: “incluir ou proteger em uma cápsula ou como em uma cápsula.”¹
- Em P00:
 - Ocultar e proteger o estado;
 - Ocultar o processo.

¹-Dicionário Michaelis Online. Disponível em < <https://michaelis.uol.com.br/busca?id=kQQD>>, acessado em Fev/2022

EX: PIZZA E ENCAPSULAMENTO

```
class Pizza {  
    public int maxIngredientes;  
    public double precoBase;  
  
    ...  
}
```

```
Pizza pizza1 = new Pizza();  
pizza1.quantidadeIngredientes = 12;  
pizza1.precoBase = 36.50;
```

Pizza
- maxIngredientes: int - precoBase: double - valorPorAdicional: double - quantidadeIngredientes: int
+ Pizza() + Pizza(adicionais: int) - init(adicionais: int) : void - valorAdicionais() : double - podeAdicionar(quantos: int) : boolean + valorFinal() : double + adicionarIngredientes(quantos: int) : int + notaDeCompra() : String

EX: PIZZA E ENCAPSULAMENTO

```
class Pizza {  
    public int maxIngredientes;  
    public double precoBase;  
  
    ...  
}
```

```
Pizza pizza1 = new Pizza();  
pizza1.quantidadeIngredientes = 12;  
pizza1.precoBase = 36.50;
```

Pizza
- maxIngredientes: int - precoBase: double - valorPorAdicional: double - quantidadeIngredientes: int
+ Pizza() + Pizza(adicionais: int) - init(adicionais: int) : void - valorAdicionais() : double - podeAdicionar(quantos: int) : boolean + valorFinal() : double + adicionarIngredientes(quantos: int) : int + notaDeCompra() : String

EX: PIZZA E ENCAPSULAMENTO

```
class Pizza {  
    public int maxIngredientes;  
    public double precoBase;  
  
    ...  
}
```

```
Pizza pizza1 = new Pizza();  
pizza1.quantidadeIngredientes = 12;  
pizza1.precoBase = 36.50;
```

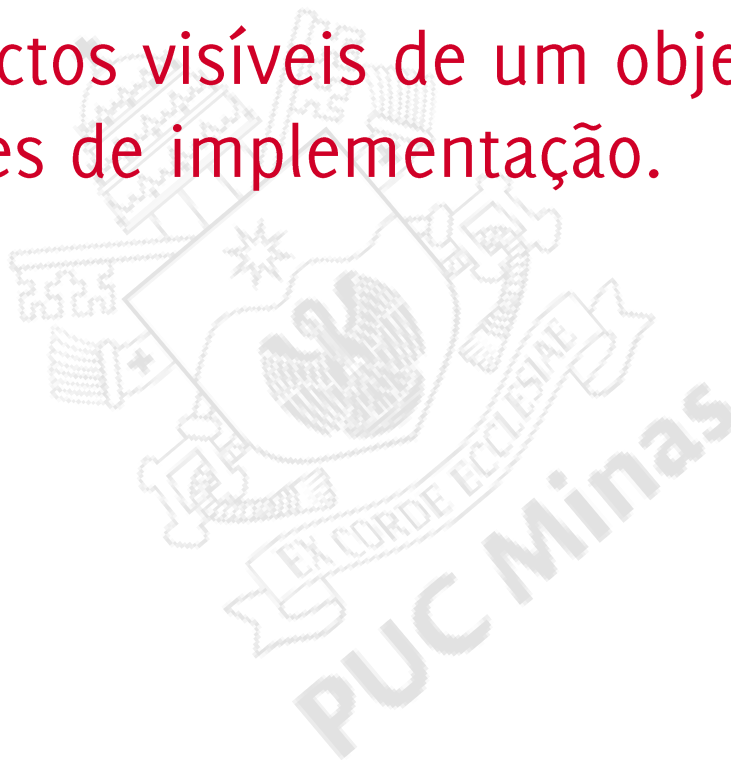
```
System.out.println(pizza1.notaDeCompra());
```

Pizza
- maxIngredientes: int - precoBase: double - valorPorAdicional: double - quantidadeIngredientes: int
+ Pizza() + Pizza(adicionais: int) - init(adicionais: int) : void - valorAdicionais() : double - podeAdicionar(quantos: int) : boolean + valorFinal() : double + adicionarIngredientes(quantos: int) : int + notaDeCompra() : String

ENCAPSULAMENTO

43

- Separa aspectos visíveis de um objeto ou classe de seus detalhes de implementação.



ENCAPSULAMENTO

44

- Separa aspectos visíveis de um objeto ou classe de seus detalhes de implementação.
- Detalhes ocultos, interface exposta.
 - Interface: aquilo que o usuário do objeto vê;
 - Interface: maneira como o usuário utiliza o objeto.

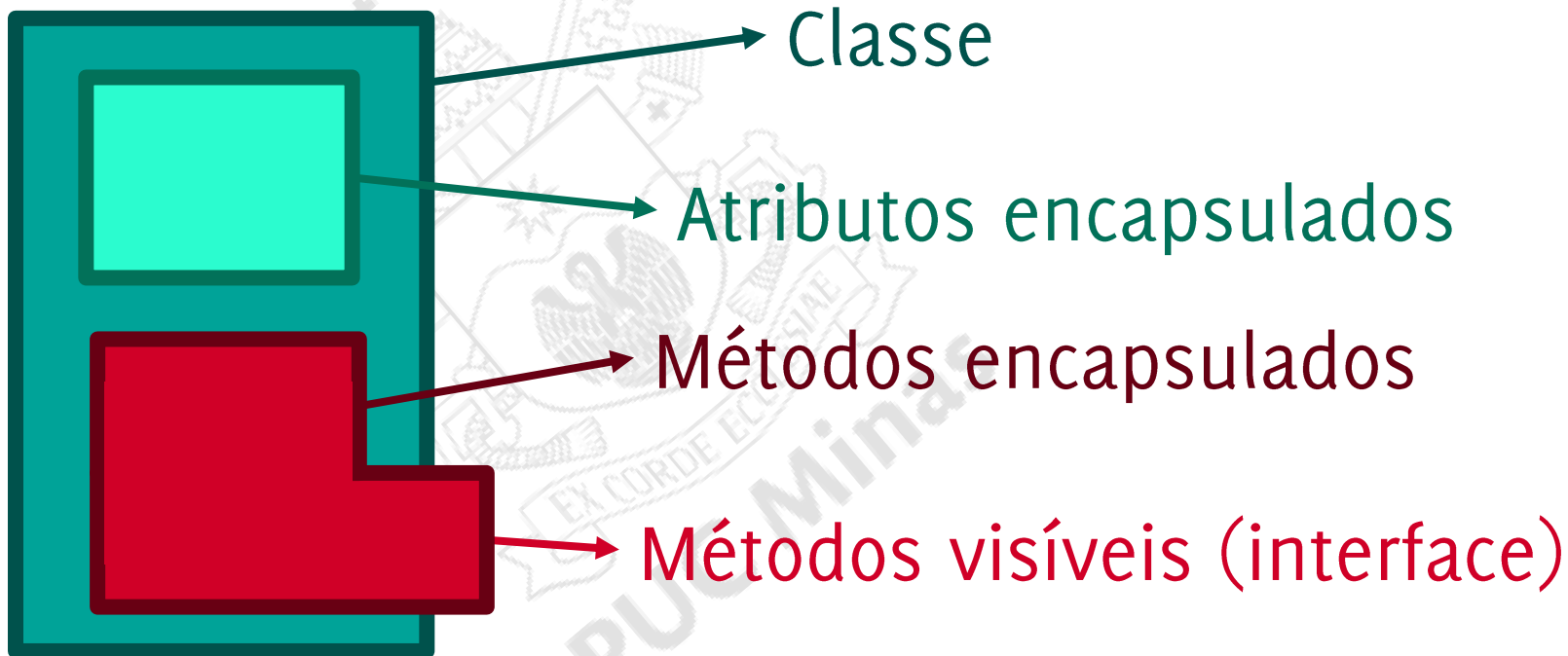
ENCAPSULAMENTO

45

- Permite alteração na implementação sem impactos em outros módulos do sistema;
- Favorece a abstração;
- Protege dados/atributos do acesso direto a partir de um código externo.

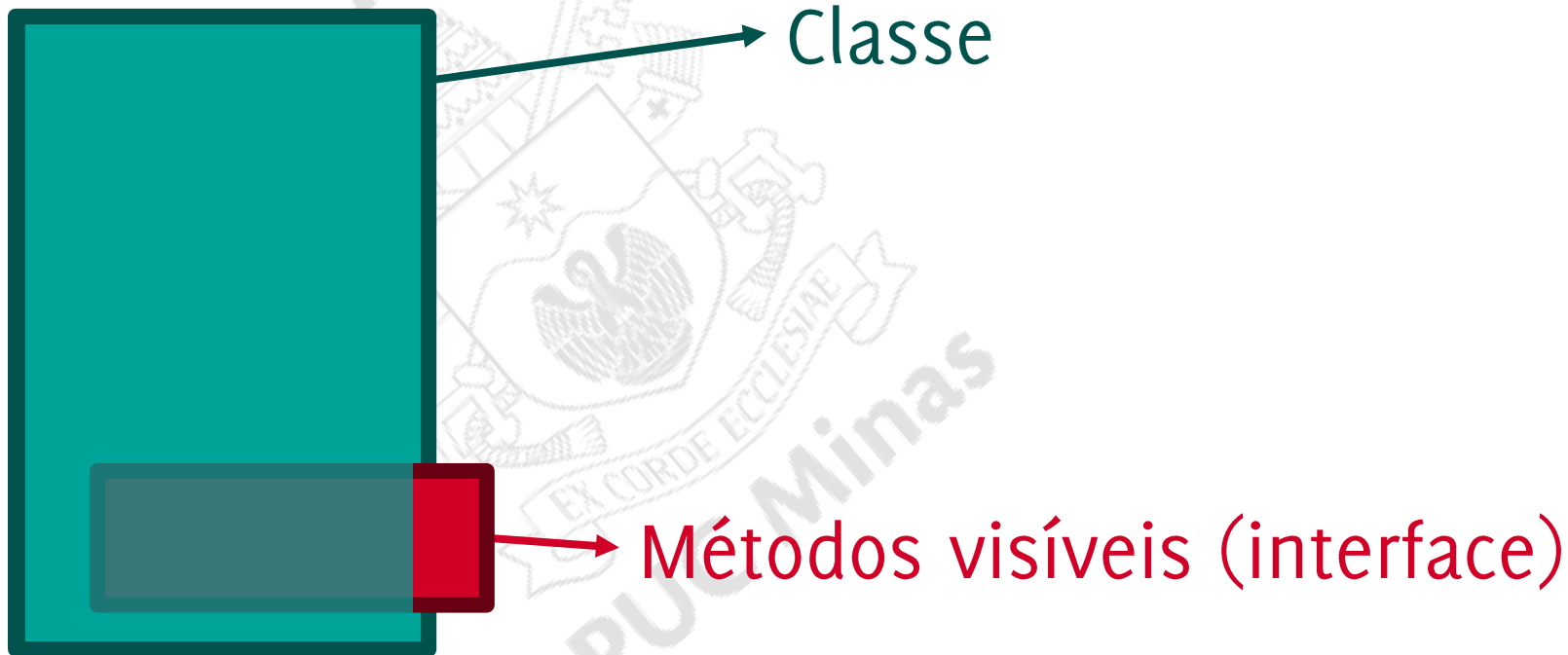
ENCAPSULAMENTO

46



ENCAPSULAMENTO

47



ENCAPSULAMENTO E PROJETO OO

48

- ▣ P00, segundo Alan Kay:
 - ▣ troca de mensagens;
 - ➔ ▣ proteção e retenção locais possibilitando ocultar o estado ou o processo;
 - ▣ associação tardia e dinâmica de tudo o que for possível.

NÍVEIS DE ACESSO E MODIFICADORES

49

- Os níveis de acesso típico em linguagens OO são:
 - Público;
 - Privado;
 - Protegido.



NÍVEIS DE ACESSO

50

- ▣  Público (acesso irrestrito):
 - ▣ Classe, atributos ou métodos visíveis em qualquer parte.
- ▣  Privado (acesso totalmente restrito):
 - ▣ Classe, atributos ou métodos visíveis somente para quem os declarou.

NÍVEIS DE ACESSO

51

- # Protegido:
 - Classe, atributos ou métodos visíveis para quem o declarou e para módulos derivados deste.
 - Aplicação na *especialização com herança* (veremos mais adiante)

{ Quero ver código!! }



ENCAPSULAMENTO

53

- Internamente, a classe pode se referir a seus componentes utilizando a palavra-chave this.
 - Clareza;
 - Desambiguação.

PIZZA

```
class Pizza {  
  
    public int adicionarIngredientes(int quantos) {  
        if(podeAlterar(quantos)){  
            quantidadeIngredientes += quantos;  
        }  
        return quantidadeIngredientes;  
    }  
}
```

PIZZA

```
class Pizza {  
  
    public int adicionarIngredientes(int quantos) {  
        if(this.podeAlterar(quantos)){  
            this.quantidadeIngredientes += quantos;  
        }  
        return this.quantidadeIngredientes;  
    }  
}
```

ENCAPSULAMENTO

56

- ▣ Os atributos de uma classe só devem ser acessados e modificados por meio de seus métodos.
- ▣ Um método de uma classe só deve existir se ele tiver relação com os atributos da classe.

ENCAPSULAMENTO

57

- ▣ De maneira geral, atributos são privados e métodos são públicos;
- ▣ Como toda regra, atenção para as exceções;

PIZZA

```
class Pizza {  
    private int maxIngredientes;  
    private double precoBase;  
    private double valorPorAdicional;  
    private int quantidadeIngredientes;  
  
    private void init(int adicionais) {  
        maxIngredientes = 8;  
        precoBase = 29d;  
        valorPorAdicional = 5d;  
        adicionarIngredientes(adicionais);  
    }  
    ...  
}
```

Pizza

- maxIngredientes: int
- precoBase: double
- valorPorAdicional: double
- quantidadeIngredientes: int

- + Pizza()
- + Pizza(adicionais: int)
- init(adicionais: int) : void
- valorAdicionais() : double
- podeAdicionar(quantos: int) : boolean
- + valorFinal() : double
- + adicionarIngredientes(quantos: int) : int
- + notaDeCompra() : String

PIZZA

```
class Pizza {  
    public int adicionarIngredientes(int quantos){  
        if(podeAdicionar(quantos)){  
            quantidadeIngredientes += quantos;  
        }  
        return quantidadeIngredientes;  
    }  
  
    public double valorFinal() {  
        return precoBase + valorAdicionais();  
    }  
}
```

PIZZA

```
class Pizza {  
    private double valorAdicionais() {  
        return quantidadeIngredientes * valorPorAdicional;  
    }  
  
    private boolean podeAdicionar(int quantos) {  
        int total = quantos + quantidadeIngredientes;  
        return (quantos >= 0 && total <= maxIngredientes);  
    }  
}
```

ENCAPSULAMENTO: APROFUNDANDO

DEBATENDO ENCAPSULAMENTO, MODIFICADORES E ACESSO

MODIFICADORES (JAVA)

62

- Diferentes linguagens usam palavras-chaves específicas, bem como podem implementar níveis particulares.
- Em Java, temos os modificadores:
 - Public, Private, Protected;
 - *Default (package).*

NÍVEIS DE ACESSO (JAVA)

63

- *Default (package):*
 - Não é necessário declarar no Java;
 - Classe, atributos e métodos visíveis para quem todos que fazem parte do mesmo pacote Java.
 - *Não há visibilidade para subclasses de outros pacotes.*

NÍVEIS DE ACESSO (JAVA)

64

MODIFICADOR	CLASSE	PACOTE	SUBCLASSE	MUNDO
public	sim	sim	sim	sim
protected	sim	sim	sim	não
default	sim	sim	não	não
private	sim	não	não	não

NÍVEIS DE ACESSO (JAVA)

65

MODIFICADOR	CLASSE	SUBCLASSE	MUNDO
public	sim	sim	sim
protected	sim	sim	não
private	sim	não	não

MÉTODOS DE ACESSO

66

- ▣ Métodos get: dão acesso ao valor de um atributo.
- ▣ Métodos set: realizam a atribuição segura de um valor a um atributo.
- ▣ Na recomendação original, em Java os getters e setters usam o prefixo e o nome do atributo encapsulado.

MÉTODOS DE ACESSO

67

▣ Ex:

`int getQuantidadeIngredientes();`
`void setQuantidadeIngredientes(int quantAdicionais);`

Pizza

- maxIngredientes: int
- precoBase: double
- valorPorAdicional: double
- quantidadeIngredientes: int

- + Pizza()
- + Pizza(adicionais: int)
- init(adicionais: int) : void
- valorAdicionais() : double
- podeAdicionarr(quantos: int) : boolean
- + valorFinal() : double
- + adicionarIngredientes(quantos: int) : int
- + notaDeCompra() : String

MÉTODOS DE ACESSO

68

▣ Ex:

int getQuantidadeIngredientes();
void setQuantidadeIngredientes(int quantAdicionais);

SERÁ?!?!?!?

Pizza

- maxIngredientes: int
- precoBase: double
- valorPorAdicional: double
- quantidadeIngredientes: int

- + Pizza()
- + Pizza(adicionais: int)
- init(adicionais: int) : void
- valorAdicionais() : double
- podeAdicionar(quantos: int) : boolean
- + valorFinal() : double
- + adicionarIngredientes(quantos: int) : int
- + notaDeCompra() : String

GETTERS AND SETTERS

69

- ▣ Debate:
 - ▣ É uma boa idéia termos indiscriminadamente métodos *getters* e *setters* na classe?

GETTERS AND SETTERS

70

▣ Why getter and setter methods are evil.

<https://www.infoworld.com/article/2073723/why-getter-and-setter-methods-are-evil.html>

▣ Getters/Setters. Evil. Period.

<https://www.yegor256.com/2014/09/16/getters-and-setters-are-evil.html>

MÉTODOS DE ACESSO

71

ATENÇÃO!!!

MÉTODOS DE ACESSO SEM NECESSIDADE
APENAS PIORAM SEU CÓDIGO.
(ENCAPSULAMENTO / ABSTRAÇÃO)

ATENÇÃO!!!

MÉTODOS DE ACESSO SEM VALIDAÇÃO NÃO
TÊM NENHUMA UTILIDADE.
(ENCAPSULAMENTO / CORREÇÃO)

OBRIGADO.

DÚVIDAS?